

AVATAR BASED AI MOCK INTERVIEW SYSTEM

**A Major Project Report Submitted in Partial Fulfilment of the Requirements for
the Degree of Bachelor of Technology in Computer Science & Engineering**

Submitted by

BISHNU CHARAN NAYAK

2321316046

OMM SUBHAM SAHOO

2201316020

CHINMAYA PRADHAN

2201316010

Under the esteemed guidance of

HOD- Mrs. ANITA BEHERA

ASSISTANT PROFESSOR (BEC)

Department of CSE



**DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING**

BHUBANESWAR ENGINEERING COLLEGE (BEC)

Affiliated to

**Biju Patnaik University of Technology, Odisha, Rourkela
MAY 2026**



Department of Computer Science & Engineering
Bhubaneswar Engineering College (BEC)

CERTIFICATE

This is to certify that the work embodied in this major project entitled “**AVATAR BASED AI MOCK INTERVIEW SYSTEM**” carried out by Chinmaya Pradhan (2201316010), Omm Subham Sahoo (2201316020) and Bishnu Charan Nayak (2321316046) is approved for the degree of Bachelor of Technology in Computer Science & Engineering of Bhubaneswar Engineering College, Bhubaneswar, Odisha.

Anita Behera

Assistant Professor & Head
Department of CSE

Anita Behera

(Guided By)
Department of CSE

External Examiner

ACKNOWLEDGEMENTS

Words cannot describe our gratitude to our major project guide, Mrs. Anita Behera (Assistant Professor), for his unwavering tolerance and constructive criticism. We also could not have completed this journey without the information and experience he shared. This initiative would not have been feasible without his constant support and motivation.

We thank Mrs. Anita Behera, Assistant Professor, Computer Science & Engineering, for her support and guidance. It is our honor to express our heartfelt gratitude to our teachers at BEC, Bhubaneswar and staff members for their wholehearted cooperation.

Date:
Place: Bhubaneswar

Omm Subham Sahoo
Regd.No:-2201316020

Bishnu Charan Nayak
Regd.No:-2321316046

Chinmaya Pradhan
Regd.No:-2201316010

DECLARATION

We certify that.

- (a) The work contained in the report is original and has been done by us under the general supervision of our supervisor.
- (b) The work has not been submitted to any other institute for any degree or diploma.
- (c) We have followed the guidelines provided by the institute in writing the report.
- (d) We have conformed to the norms and guidelines given in the ethical code of conduct of the institute.
- (e) Whenever we have used materials (data, theoretical analysis, and text) from other sources, we have given due credit to them by citing them in the text of the report and giving their details in the references.
- (f) Whenever we have quoted written materials from other sources, we have put them under quotation marks and given due credit to the sources by citing them and providing the required details in the references.

Date:

Place: Bhubaneswar

Omm Subham Sahoo

Regd.No:-2201316020

Bishnu Charan Nayak

Regd.No:-2321316046

Chinmaya Pradhan

Regd.No:-2201316010

ABSTRACT

This project presents the implementation of an **Avatar-Based AI Mock Interview System** that provides an interactive platform for practicing job interviews. The system takes dynamic user inputs such as resume, job description, and difficulty level to generate personalized interview questions using the Gemini 2.5 language model.

Voice-based interaction is enabled through Google Web Speech API for speech recognition and Edge TTS for voice output, creating a natural conversation flow. A 3D RPM avatar acts as the virtual interviewer, delivering questions and responses in real time. The system evaluates the user's answers based on relevance, fluency, and tone, and provides instant feedback for improvement.

This solution offers a complete mock interview experience, helping users to enhance their communication skills and prepare effectively for AI-led interviews through a lifelike and responsive virtual environment.

Table of Contents

CERTIFICATE.....	ii
DECLARATION	iii
ACKNOWLEDGEMENTS.....	iv
ABSTRACT.....	v
LIST OF FIGURES	ix
LIST OF ABBREVIATIONS.....	x
CHAPTER 1.....	1
INTRODUCTION.....	1
1.1 Introduction.....	1
1.2 Objective	2
1.3 Challenges.....	2
CHAPTER 2.....	3
LITERATURE SURVEY	3
2.1 Introduction.....	3
2.2 Literature Review	3
CHAPTER 3.....	5
PROBLEM IDENTIFICATION	5
3.1 Problem Identification	5
3.2 Problem Statement	6
3.3 Existing System	6
3.3.1 Disadvantage	7
3.4 Proposed System	7
3.4.1 Input Stage	8
3.4.2 Voice Interaction Stage	9
3.4.3 AI Response generation Stage	9
3.4.4 Text to Speech and Animation Sync Stage	10
3.4.5 3D Avatar Output Stage.....	10
3.4.6 Chat view and Response Stage	10
3.5 Advantages	11
3.6 Feasibility Study.....	11

3.6.1 Economic Viability	11
3.6.2 Technical Viability	12
3.6.3 Social Viability	12
3.7 REQUIREMENTS.....	12
3.7.1 Software Requirements	13
3.7.2 Hardware Requirements	13
CHAPTER 4.....	15
SYSTEM DESIGN.....	15
4.1 System Architecture.....	15
4.2 Diagrams.....	17
4.2.1 Use Case Diagram.....	17
4.2.2 Activity Diagram.....	18
4.2.3 Data Flow Diagram.....	20
4.2.4 Sequence Diagram	22
4.2.5 Class Diagram	23
CHAPTER 5.....	26
IMPLEMENTATION	26
5.1 Modules.....	26
5.1.1 User Management Module.....	26
5.1.2 Interview Session Module	26
5.1.3 LLM Service Module	27
5.1.4 Speech Recognition (STT) Module.....	27
5.1.5 Text to Speech (TTS) Module.....	28
5.1.6 Avatar Interaction Module	28
5.1.7 Follow Up & Generation Module.....	29
5.1.8 Evaluation and Feedback Module.....	29
5.2 Technologies Used	28
5.2.1 Large Language Model (LLM) Gemini-2.0-flash	30
5.2.2 Speech-to-Text Google WebSpeech API.....	30
5.2.3 Text-to-Speech Edge TTS.....	30
5.2.4 Avatar System Ready Player Me Avatar (RPM)	30
5.2.5 Frontend Technologies – React.js and Three.js.....	30

5.2.6 Backend Technologies Python and FastAPI	31
CHAPTER 6.....	32
TESTING.....	32
6.1 TYPES OF TESTING	32
6.1.1 Resume, Job Description, Difficulty Input	32
6.1.2 Interview Panel Functionality and Question Delivery	33
6.1.3 Voice Response and STT Accuracy	34
6.1.4 Results Evaluation and Score Display.....	35
CHAPTER 7.....	36
RESULTS	36
CHAPTER 8.....	39
CONCLUSION	39
CHAPTER 9.....	40
FUTURE WORK.....	40
CHAPTER 10.....	42
REFERENCES.....	42

LIST OF FIGURES

Figure 1: Statistics of the Job dependency on AI and failure of jobs.	1
Figure 2: System Architecture	15
Figure 3: Use Case Diagram	17
Figure 4: Activity Diagram	19
Figure 5: Data Flow Diagram Level-1	20
Figure 6: Data Flow Diagram Level-2.....	21
Figure 7: Sequence Diagram	22
Figure 8: Class Diagram	24
Figure 9: Input section Test results.....	30
Figure10: Interview panel test results.....	31
Figure 11: Interview Results report test results.	33
Figure 12: Input section results.....	34
Figure 13: Interview Panel results	35
Figure 14: Results Pallet results.....	36

LIST OF ABBREVIATIONS

AI	-	Artificial Intelligence
LLM	-	Large Language Model
STT	-	Speech-to-Text
TTS	-	Text-to-Speech
JD	-	Job Description
UI	-	User Interface
UX	-	User Experience
API	-	Application Programming Interface
RPM	-	Ready Player Me
GPT	-	Generative Pre-trained Transformer
PDF	-	Portable Document Format
DOCX	-	Microsoft Word Open XML Document

CHAPTER 1

INTRODUCTION

1.1 Introduction

The recruitment industry has undergone a major transformation in recent years, with Artificial Intelligence (AI) taking centre stage in various stages of the hiring process. One of the most notable changes is the rapid adoption of **AI-based interview systems** by companies across the globe. These systems offer speed, consistency, and cost-efficiency, making them an attractive choice for modern recruiters. However, for candidates, this shift has introduced new challenges.

According to industry trends, the adoption rate of AI interviews has increased steadily from around **10% in 2019** to an estimated **65% by 2024**. Alongside this, the **candidate failure rate** in AI interviews has also seen a significant rise — from just **5% to nearly 38%** — highlighting the growing gap between traditional interview preparedness and AI-based expectations.

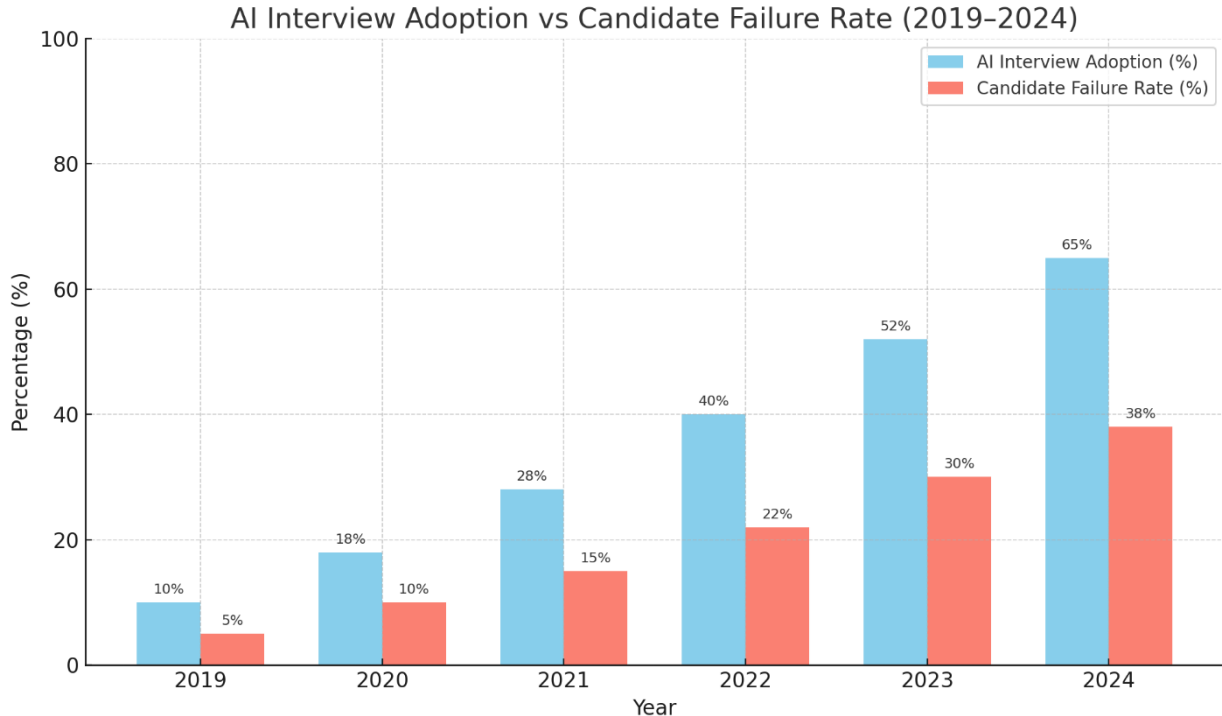


Figure 1: Statistics of the Job dependency on AI and failure of jobs

1.2 Objective

The primary objective of this project is to design and implement a realistic AI-based mock interview simulator that allows users to practice and prepare for modern AI-driven interviews. The system aims to generate dynamic and role-specific interview questions by accepting user inputs such as resume details, job description, and desired difficulty level. It leverages the Gemini 2.5 Flash model for natural language interactions, enabling a human-like experience through an avatar-based interface. Additional goals include integrating voice-based communication using speech recognition (Google Web Speech API) and text-to-speech (Edge TTS) for a smooth conversational flow. Overall, the project intends to help candidates gain confidence, improve communication, and adapt to the evolving landscape of AI interviews.

1.3 Challenges

While building the AI-based mock interview simulator, we faced several technical and integration-related challenges. These issues impacted both the development process and the overall user experience of the system.

- **Token Limitation of Gemini 2.5 Flash:** Since we used the free version of the model, there were restrictions on the number of tokens that could be processed. This limited the length and complexity of generated interview questions and responses.
- **Latency Issues:** Delays were observed while generating responses from the model and during speech processing, which affected the real-time conversation experience.
- **Speech Recognition Errors:** The Google Web Speech API occasionally struggled to capture user input accurately in noisy environments or with varied accents, leading to miscommunication.
- **Robotic or Delayed TTS Output:** Although Edge TTS was used for generating responses, it sometimes produced robotic or slightly delayed audio, affecting the natural flow of conversation.
- **Integration Complexity:** Coordinating all components—Gemini model, speech recognition, text-to-speech, and RPM avatar—into a smooth, unified system required detailed planning and debugging due to compatibility and timing issues.
- **Avatar Synchronization:** Ensuring that the RPM avatar's movements were synced with the TTS output was challenging, especially while maintaining a realistic visual experience.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

Traditional interview preparation tools such as fixed question banks and manual mock interviews often fail to simulate real-world complexity, provide personalized feedback, or maintain user engagement. These systems lack adaptability, interactivity, and emotional intelligence, limiting their effectiveness for candidates preparing for competitive job markets.

With the advent of **Artificial Intelligence (AI)** and **3D technologies**, new approaches have emerged. These include **Large Language Models (LLMs)** for natural language understanding, **Text-to-Speech (TTS)** and **Speech-to-Text (STT)** systems for voice communication, and **3D avatars** for virtual presence. By integrating these components, researchers are working toward creating intelligent and interactive platforms for training, skill assessment, and soft-skill development.

The **Avatar-Based AI Mock Interview System** leverages this technological evolution by simulating a lifelike interview scenario where users interact with a virtual 3D interviewer that not only evaluates their responses but also provides personalized feedback in real time. This chapter discusses several research efforts that contribute to or inspire components of this project.

2.2 Literature Review

1. **Zohaib Hassan et al. (2022)** proposed a VR-based talking avatar system aimed at training child protection professionals. The system was built using Unity 3D and incorporated RASA for natural language understanding and IBM Watson services for speech synthesis and recognition. The avatar was animated using the Salsa Suite for lip-sync and the Unity Multipurpose Avatar (UMA) framework for realistic modeling. Their study demonstrated that such interactive and immersive systems improve user engagement, training effectiveness, and realism in simulated interviews.

2. **EZInterviewer (2023)** introduced a transformer-based mock interview generator that

creates personalized interview questions using models like BERT. It utilizes user inputs such as resumes and job descriptions to generate relevant dialogue. While this system operates primarily in a text-based format, it highlighted the importance of dynamic question generation and personalized interaction in the interview training domain.

3. **Microsoft Research** developed neural TTS systems, such as those used in Azure TTS, capable of generating highly realistic and natural-sounding synthetic voices. These have been used in applications like digital assistants and conversational bots. The focus of these systems is on enhancing user experience through lifelike speech, which plays a key role in making virtual interactions more human-like and engaging.
4. **OpenAI and Google DeepMind** have released powerful language models such as GPT-4 and Gemini 2.0 Flash, which are capable of generating coherent multi-turn conversations with contextual memory. These models support real-time interaction and can understand the user's sentiment and intent to provide more personalized and human-like responses in dialogue systems.
5. **Adobe Mixamo and Rhubarb Lip Sync** are widely used tools for creating animated 3D avatars. Mixamo offers full-body animations, while Rhubarb enables accurate lip-syncing by analyzing audio input and generating corresponding mouth movements. These tools are valuable in bringing avatars to life, making them expressive and synchronized with speech during conversations.
6. **Google Web Speech API** offers robust speech recognition capabilities, converting spoken language into text in real-time. This API is often used in web applications to enable voice-based interaction. Its reliability and ease of integration make it a popular choice for applications requiring hands-free communication.
7. **Research on emotion recognition in virtual agents** has shown that avatars capable of expressing emotions through facial expressions, tone, and gestures contribute to improved user experience. These features enhance engagement and provide deeper insights into user behavior, especially in training and educational simulations.

CHAPTER 3

PROBLEM IDENTIFICATION

3.1 Problem Identification

Preparing for job interviews is a critical phase in a candidate's career journey, yet it is often marred by anxiety, lack of real-time feedback, and insufficient practice in realistic scenarios. Traditional mock interview methods—whether conducted manually by peers, mentors, or through question banks—fail to offer the dynamic, interactive, and personalized experience that real interviews demand. These conventional methods lack contextual understanding, emotional intelligence, and adaptability, leaving candidates ill-prepared for the unpredictable nature of actual interviews.

One of the primary challenges lies in the absence of systems that can provide real-time, voice-based, and context-aware feedback on performance. Candidates often struggle to assess their communication effectiveness, body language, tone, and relevance of answers without an intelligent, unbiased evaluator. Moreover, with job roles and industries becoming increasingly diverse, a one-size-fits-all interview simulator does not meet individual preparation needs.

To overcome these limitations, this project proposes the development of an AI-powered 3D avatar mock interview system. The system aims to simulate lifelike interviews by leveraging Gemini 2.0 Flash, a high-speed large language model, in combination with speech-to-text (STT), text-to-speech (TTS), and real-time 3D animation technologies. By analysing the user's speech, tone, and content, the avatar can dynamically generate relevant follow-up questions, deliver voice responses, and provide performance evaluations in a fully interactive environment.

The challenge is to ensure the avatar's behaviour feels natural and human-like through accurate lip-syncing, expressive gestures, and intelligent response generation. Furthermore, the system must handle variations in user speaking style, background noise, and question context without compromising accuracy or performance.

This project addresses the problem of ineffective and outdated interview preparation methods by introducing a non-intrusive, immersive, and AI-driven solution. It aims to equip users

with a smarter, scalable, and realistic training experience that not only reduces anxiety but also builds confidence and improves communication skills—ultimately enhancing interview performance and employability.

3.2 Problem Statement

Facing job interviews can be an overwhelming experience for many candidates, especially when traditional preparation methods fall short of offering practical exposure and real-time evaluation. Static question banks, scripted mock interviews, or mentor-led sessions do not capture the complexity, pressure, or dynamism of actual interviews. These methods lack emotional intelligence, adaptability, and the ability to provide personalized feedback on communication skills, confidence, tone, and body language. As industries evolve and interview formats diversify, candidates need more than generic practice—they need contextual, immersive, and feedback-driven preparation. This gap calls for an advanced system that mimics real interview scenarios while offering constructive, real-time insights. Our proposed solution is a 3D avatar-based AI mock interview system that enables users to interact naturally with a lifelike virtual interviewer. By leveraging speech recognition, text-to-speech synthesis, large language models like Gemini 2.0 Flash, and 3D animation technologies, the system can conduct dynamic interviews, analyze user responses, and provide intelligent feedback. This ensures users receive a more authentic, interactive, and confidence-boosting experience, better preparing them for real-world opportunities.

3.3 Existing System

In recent years, several intelligent interview simulation platforms and virtual avatar-based systems have been proposed to improve training and evaluation in communication-heavy domains. One notable example is the **AI-Driven Talking Avatar in Virtual Reality** developed for training Child Protection Services (CPS) professionals. This system was designed to simulate investigative interviews with child avatars using **Virtual Reality (VR)**, **dialogue models**, and **speech technologies**.

The system used **RASA** for NLP-based dialogue generation, **Unity 3D** for rendering avatars in a VR environment, and **IBM Watson's STT and TTS APIs** for real-time speech processing. Avatars were animated using **UMA (Unity Multipurpose Avatar)** and **Salsa Suite** to provide lip-syncing and gesture animations that enhanced realism and user immersion. The backend included pre-scripted dialogue models trained on real interview transcripts, while the

front-end was accessible via Oculus Quest 2 VR headsets. The interaction followed a structured protocol, and users were required to press buttons to move through conversation turns. While the system showed promising results in training professionals, it relied heavily on VR devices, predefined dialogue structures, and manual navigation.

Though effective in specific domains like child protection, these systems present certain limitations when considered for job interview preparation:

3.3.1 Limitations:

- Most systems are **domain-specific** and not designed for generalized use cases like corporate interviews.
- **VR dependency** limits accessibility and scalability for everyday users.
- Dialogue models used in earlier systems are often **rule-based or semi-flexible**, lacking deep contextual adaptation and spontaneity.
- Speech interaction required button-pressing for transitions, reducing natural conversational flow.
- **Lack of feedback mechanisms** like performance scoring or sentiment analysis makes it difficult for users to track progress.

3.4 Proposed System

To overcome the limitations of traditional interview preparation methods, we propose an **Avatar-Based AI Mock Interview System** that offers a realistic, interactive, and intelligent mock interview experience. This system leverages **Large Language Models (LLMs)**, **3D avatar technology**, and **real-time voice processing** to simulate a lifelike interview scenario.

The system architecture is designed to integrate various AI technologies seamlessly. The user begins by uploading their **resume (PDF)**, **job description**, and selecting a **difficulty level**. They then interact with the interviewer avatar using **live voice input** via a microphone. The user's responses are processed in real-time by the **Speech-to-Text (STT)** module, converting the speech into text.

Once transcribed, the **Natural Language Processing (NLP)** module and **Gemini 2.0 Flash (LLM)** analyze the response, generate dynamic follow-up questions, and evaluate the

content based on multiple factors like **relevance, tone, confidence, and timing**. Simultaneously, the system uses **Text-to-Speech (TTS)** technology to vocalize the avatar's questions and responses, ensuring smooth and natural voice communication.

To enhance realism, the avatar uses **Rhubarb Lip Sync** to synchronize lip movements with audio. The avatar appears on the user interface built with **React.js**, offering a lifelike interviewer experience. The **chat view panel** displays a transcript of the conversation for post-interview analysis.

At the end of the session, the system generates a **performance evaluation report**. This includes scores and feedback based on communication skills, confidence level, and technical relevance. The evaluation helps users identify areas of improvement and track progress over time.

This proposed system aims to create a **non-intrusive, immersive, and AI-driven interview training environment** that helps users overcome anxiety, refine communication skills, and build confidence for real job interviews.

In the proposed methodology, there are four stages.

- Input Collection Stage
- Voice Interaction Stage
- AI Response Generation Stage
- Text-to-Speech and Animation Sync Stage
- 3D Avatar Output Stage
- Chat View and Response Logging

3.4.1 Input Collection Stage

The first stage of the system begins with the collection of essential data from the user. This includes uploading their **resume (in PDF format)**, optionally providing a **job description**, and selecting the **difficulty level** of the mock interview (such as easy, medium, or hard). These inputs are crucial as they help personalize the experience for the user. The resume provides information about the candidate's qualifications, skills, and background, while the job description outlines the role they are targeting. The selected difficulty level determines the complexity and depth of the questions asked by the system.

This input data is parsed and passed to the backend, where it guides the AI in generating relevant and context-specific interview questions. The goal of this stage is to create a mock interview environment that closely reflects the user's real-world goals. By customizing the interaction based on individual input, the system ensures that the generated questions are not only relevant but also practical, giving the user targeted practice instead of generic responses.

3.4.2 Voice Interaction Stage

Once the setup is complete, the user can begin the interview by **speaking into a microphone**. This is where the hands-free and natural interaction begins. The spoken input is captured in real-time and passed through a **Speech-to-Text (STT)** module (e.g., Google Web Speech API), which transcribes the audio into text. This allows the system to process and analyze user responses using AI, removing the need for typing and enhancing realism.

This stage replicates how real interviews function—via voice—and introduces the challenge of clear and confident verbal communication. Since interviews are usually time-sensitive and require immediate thinking, this voice-based input system prepares users to answer questions naturally. It also sets the foundation for the avatar to respond appropriately and for the AI to interpret the tone, choice of words, and fluency of the candidate.

3.4.3 AI Response Generation

After the user's response is converted to text, the **Large Language Model (LLM)**—such as Gemini 2.0 Flash or LLaMA 3.3—comes into play. It processes the input using Natural Language Processing (NLP) techniques and dynamically generates an intelligent and appropriate follow-up question or response. The LLM also considers the resume, job description, and previous dialogue to ensure the conversation is **context-aware and continuous**.

This stage is the "brain" of the system. It makes the avatar seem intelligent and aware of the flow of the interview. Unlike static question banks or rule-based systems, this model understands what the user said and tailors its next response accordingly. The AI can evaluate whether the candidate answered correctly, skipped key details, or sounded uncertain, and then ask deeper or clarifying questions to simulate a real interviewer's behaviour.

3.4.4 Text-to-Speech and Animation Sync

Once the AI has generated the next response, it is passed to the **Text-to-Speech (TTS)** engine (like Edge TTS) to convert it into realistic, human-like voice. This voice output is then

prepared to be delivered through a 3D avatar. However, to make the avatar's speech look real, we also use **Rhubarb Lip Sync**, which analyses the spoken audio and generates precise **lip movement data**.

In parallel, the system uses **Adobe Mixamo** to trigger natural body movements and gestures like nodding, hand gestures, or head tilts. These animations, combined with accurate lip sync, make the 3D avatar appear lifelike and professional. This synchronization step is what sets this system apart from traditional chatbots or text-based interview tools—it offers visual realism and believable human interaction.

3.4.5 3D Avatar Output

This stage brings all the earlier components together in a visually engaging experience. The 3D avatar—created using tools like Blender and Mixamo—appears on screen as the interviewer. It speaks the AI-generated questions using TTS voice and performs lip-synced animations, making the interaction feel like a real video call or in-person interview. This level of realism helps the user become comfortable with visual cues and body language.

The avatar serves as a virtual representation of an interviewer, which can improve the user's ability to handle pressure, interpret non-verbal signals, and practice speaking confidently in front of a person-like figure. Over time, this exposure can help users overcome interview anxiety and improve verbal communication and eye contact, even though the avatar is digital.

3.4.6 Chat View and Response Logging

Throughout the interaction, every question and response is recorded in a **chat view panel**. This chat interface serves as a transcript of the entire mock interview session. It allows the user to scroll back and review how they answered, what kind of follow-ups were asked, and where they might have hesitated or repeated themselves.

This stage also supports learning after the session is over. Users can go through their chat history to analyze mistakes, see how questions evolved, and reflect on areas needing improvement. The system can later use this log to generate a **detailed performance report** that includes feedback on tone, confidence, keyword matching with the job description, and suggestions for improvement.

3.5 Advantages

- Simulates a real-time interview experience using a 3D avatar
- Enhances candidate preparation with realistic interviewer responses
- Personalized interviews based on resume, job role, and difficulty level
- Supports voice input and generates natural voice output using TTS
- Uses large language models (LLMs) for smart and context-aware responses
- Integrates lip sync and animation for a life-like virtual avatar
- Reduces the need for human resources during mock interviews
- Available anytime, improving accessibility and flexibility for users
- Encourages candidates to practice and improve communication skills

3.6 Feasibility Study

This stage evaluates whether the proposed 3D AI-Based Virtual Interviewer System can be practically implemented from an economic, technical, and social perspective. It also involves analyzing the system's impact, cost-effectiveness, ease of deployment, and user acceptance. The feasibility analysis ensures that the system provides value without becoming a burden in terms of resources or complexity. A high-level implementation plan, cost forecast, and technical analysis have been considered to validate the system's potential.

3.6.1 Economic Viability

The proposed system has been developed using a cost-effective approach. Most technologies used—such as React.js, Python, Blender, Rhubarb, Mixamo, FastAPI, and open-source LLM integrations—are freely available for development and testing. There is no need for expensive hardware or licensing. Since the application is web-based, deployment costs are minimal. Only optional upgrades, such as VR integration or custom TTS voices, might incur extra charges in the future. Overall, the system is budget-friendly and suitable for deployment in institutions, startups, or individual users preparing for job interviews.

3.6.2 Technical Viability

The system is technically feasible, as it utilizes well-supported and commonly used technologies that are compatible across platforms. It does not require any specialized hardware apart from a microphone and a standard webcam-enabled system. The backend, built with Python

and FastAPI, communicates efficiently with the frontend React interface. Speech recognition, avatar rendering, and AI processing can be run on standard computing systems. The system has been designed to run in browsers, which eliminates the need for additional installations and keeps the technical demand low. This ensures smooth deployment in training centers, colleges, and homes.

3.6.3 Social Viability

The proposed system is designed to be user-friendly, non-intrusive, and supportive of personal development. Users interact naturally via voice and receive feedback in a familiar interview-like format. The 3D avatar provides a human-like interface, reducing the anxiety often experienced in traditional mock interviews. The chat view and feedback system help users identify areas of improvement, boosting confidence and acceptance. The system can be easily adopted by students, job seekers, and training professionals. Minimal training is needed to operate the interface, and its interactive nature encourages repeat usage and positive word-of-mouth adoption.

3.7 REQUIREMENTS

The design and development of the 3D AI-Based Virtual Interviewer System focused on creating a seamless, interactive, and user-friendly experience. Navigation between modules such as the avatar, chat view, and input/output screens was kept smooth and intuitive. The application was also designed to minimize manual input by the user, using voice-based interaction and AI-driven automation wherever possible.

3.7.1 Software Requirements

For developing and running the application, the following software components are required:

- **Operating System:**
 - Windows 10 / Windows 11 (64-bit)
 - Ubuntu 20.04+ (for deployment, optional)

- **Programming Languages and Frameworks:**

- Python 3.9+
- FastAPI (Backend API)
- React.js (Frontend Interface)
- LangChain (LLM integration)
- JavaScript / HTML / CSS

- **Libraries and APIs:**

- Google Web Speech API (Speech-to-Text)
- Edge TTS API (Text-to-Speech)
- Rhubarb Lip Sync (Lip movement sync)
- Adobe Mixamo (3D animations)
- Blender (Avatar modeling and rigging)

- **Tools and Platforms:**

- Visual Studio Code / PyCharm
- Docker (optional for container deployment)
- Git (Version Control)

3.7.2 Hardware Requirements

To develop and run the system effectively, the following hardware configuration is recommended:

- **Processor:** Intel Core i5 or higher (Quad-Core)
- **RAM:** Minimum 8 GB (16 GB recommended for smooth rendering and LLM processing)
- **Hard Disk:** 512 GB SSD or higher
- **GPU (Optional but recommended):** NVIDIA GTX 1050 or higher for 3D rendering and smooth avatar animations
- **Peripherals:**
 - Microphone (for voice input)
 - Webcam (optional, for future avatar mirroring features)

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The system architecture of the 3D AI-based Virtual Interviewer is divided into two major components: **Frontend** and **Backend**, which work together to create a seamless, real-time interview experience for the user.

The **Frontend** is responsible for taking user inputs and displaying outputs. The user begins by uploading their **resume (PDF)**, **job description**, and selecting a **difficulty level**. These inputs are used to guide the virtual interviewer's questioning. Additionally, a microphone is activated to allow the user to respond to questions through voice input.

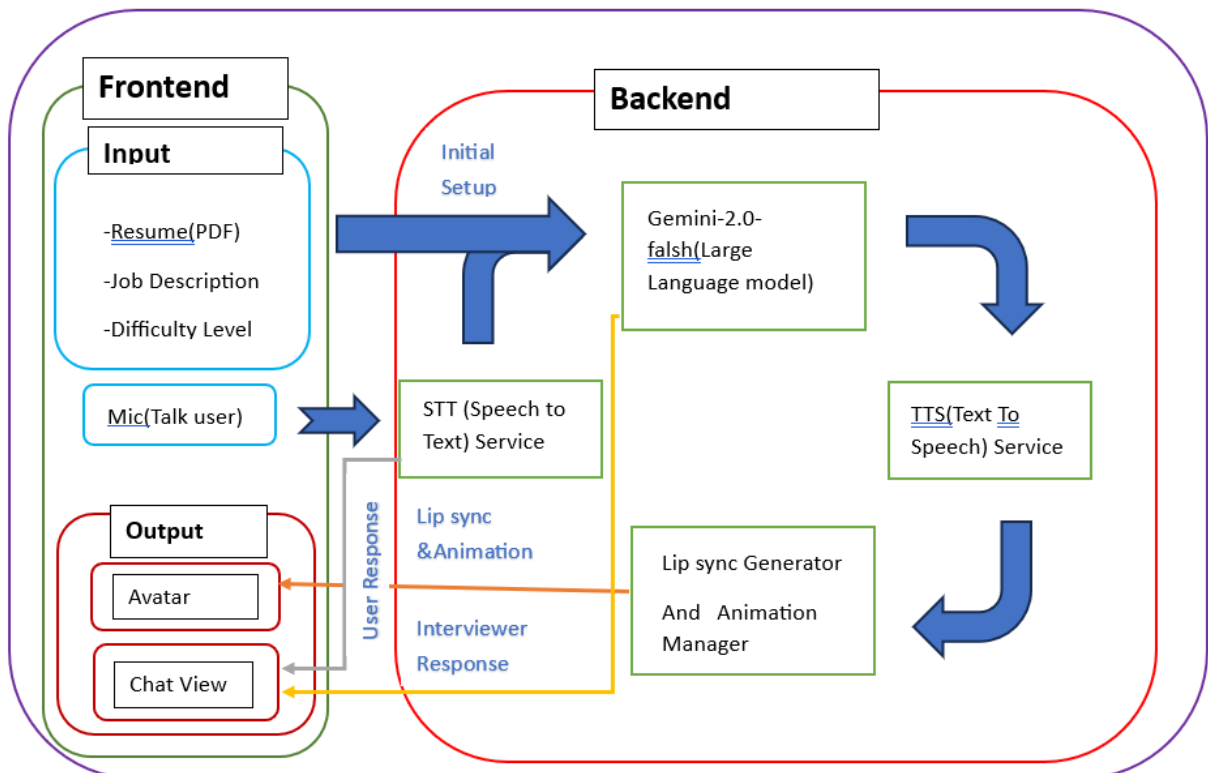


Figure 2: System Architecture

The user's voice is sent to the **Speech-to-Text (STT) service**, which transcribes the spoken words into text. This transcribed response is then sent to the **backend**, where a **large language model (Gemini 2.0 - Flash)** processes the input and generates a suitable interviewer response based on the resume, job description, and difficulty level provided earlier.

The generated text response is passed to a **Text-to-Speech (TTS) service**, which converts it into synthetic speech. Simultaneously, the **Lip Sync Generator and Animation Manager** module creates corresponding facial animations for the 3D avatar by mapping the TTS output to mouth movements.

Finally, the **Frontend** receives both the audio and animation data. The 3D interviewer avatar is animated accordingly, and the spoken response is delivered to the user. A **chat view panel** also displays the textual version of the conversation for better clarity and accessibility.

This client-server interaction ensures a lifelike, intelligent virtual interview experience by combining voice interaction, animation, and natural language understanding in real time.

- **Input Handling (Frontend):**

The system starts by collecting user inputs through the interface. The user uploads their **resume (PDF)**, **job description**, and selects the **interview difficulty level**. Additionally, a **microphone input** is used to capture the user's verbal responses during the interview.

- **Speech-to-Text (STT) Service:**

The user's voice input is passed to a **Speech-to-Text service**, which converts the spoken words into textual format for further processing. This is a crucial step that bridges the communication between the user and the backend model.

- **AI Interviewer Response Generation:**

The transcribed user response is processed by a **Large Language Model (Gemini 2.0 - flash)** in the backend. It evaluates the response based on the provided resume, job description, and difficulty level, and generates a relevant follow-up question or feedback as the interviewer.

- **Text-to-Speech (TTS) Service:**

The generated text response from the language model is passed to the **Text-to-Speech engine**, which converts it into a human-like voice to be spoken by the avatar.

- **Lip Sync and Animation:**

Simultaneously, the **Lip Sync Generator and Animation Manager** convert the TTS output into synchronized facial animations. This makes the virtual avatar look more realistic while speaking, enhancing the immersion.

- **Output Display (Frontend):**

The final animated avatar, along with the spoken audio and chat view, is rendered on the frontend. The **avatar visually presents the question** and **chat view displays the text**, giving the user a complete, real-time virtual interview experience.

4.2 Diagrams

4.2.1 Use Case Diagram

This system simulates an AI-powered mock interview using avatar interaction, real-time speech recognition, AI-generated questions, and feedback mechanisms.

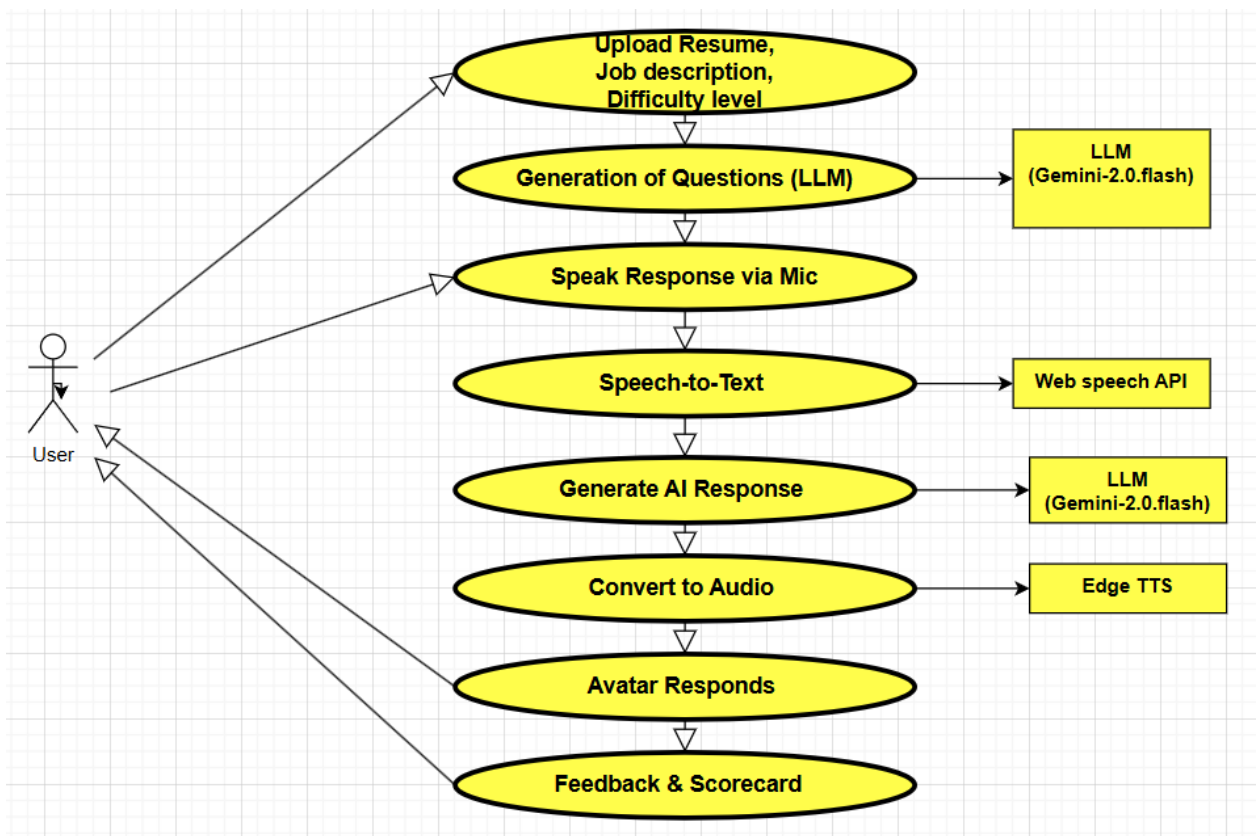


Figure 3: Use Case Diagram

Actors

- The User is the central actor interacting with the system.

Key Use Cases

- The user uploads a resume, job description, and selects the difficulty level.
- The system generates interview questions using a large language model (LLM).
- The user responds to the questions using a microphone.
- The system converts the user's spoken response to text.
- The LLM processes the response and generates follow-up questions or evaluations.
- The generated text is converted to audio.
- A 3D avatar speaks the AI-generated audio response with lip-sync.
- The user receives feedback and a scorecard.

External Components

- Gemini-2.0.flash is used for generating questions and AI responses.
- Google Web Speech API is used for speech-to-text conversion.
- Edge TTS is used for text-to-speech conversion.
- A 3D avatar component is used for response visualization.

Step-by-Step Flow

1. The user starts by uploading the resume, job description, and choosing the interview difficulty.
2. These inputs are passed to the LLM to generate a set of relevant interview questions.
3. The user speaks their answer through a microphone.
4. The speech is captured and converted to text using a speech recognition API.
5. The text is passed to the LLM to generate a follow-up response or evaluation.
6. The response is then converted into audio using a TTS system.
7. The avatar presents the audio response with synchronized lip movement.
8. Finally, the user receives an evaluation of their answer along with a feedback scorecard.

4.2.2 Activity Diagram

The activity diagram represents the flow of actions taken by the user and system from start to finish during an AI-powered mock interview session.

Flow of Activities

- The process begins when the user starts the system.
- User uploads a resume.
- User provides a job description.

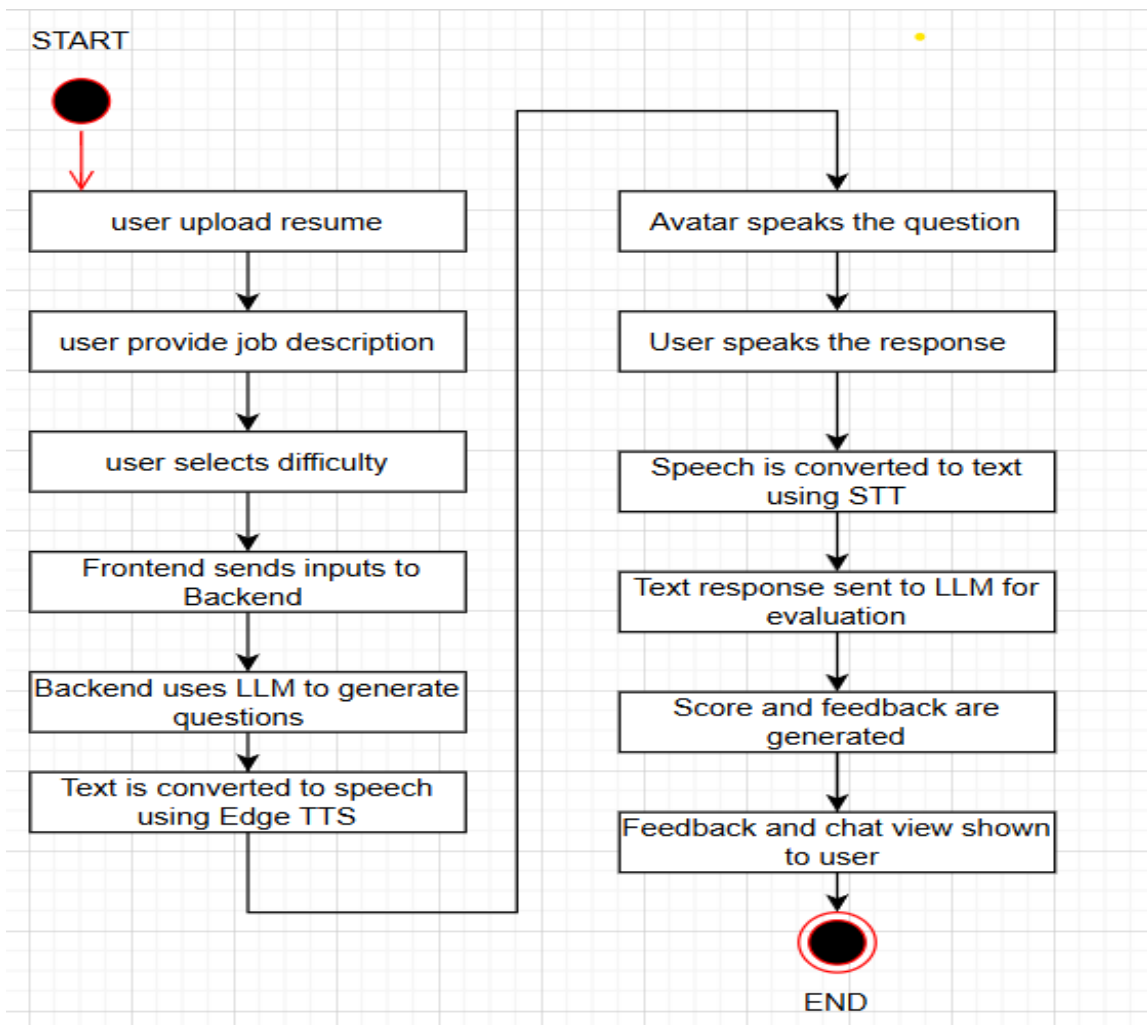


Figure 4: Activity Diagram

- User selects the difficulty level for the mock interview.
- Frontend collects all inputs and sends them to the backend server.
- Backend uses an LLM (such as Gemini-2.0.flash) to generate relevant interview questions.
- The generated questions are converted to speech using a text-to-speech (TTS) system like Edge TTS.
- The avatar presents the generated question to the user.
- The user speaks their response using a microphone.
- The system captures the user's audio and converts it to text using a speech-to-text (STT) engine.
- The transcribed response is sent to the LLM for evaluation.
- The LLM analyzes the response and generates a score and feedback.
- The system shows the feedback and scorecard to the user in a chat view.

- The session ends.

4.2.3 Data Flow Diagrams

Data Flow Diagram – Level 1 (Context Level)

This L1 diagram gives a **high-level overview** of how data flows in the *Avatar-Based AI Interview System*.

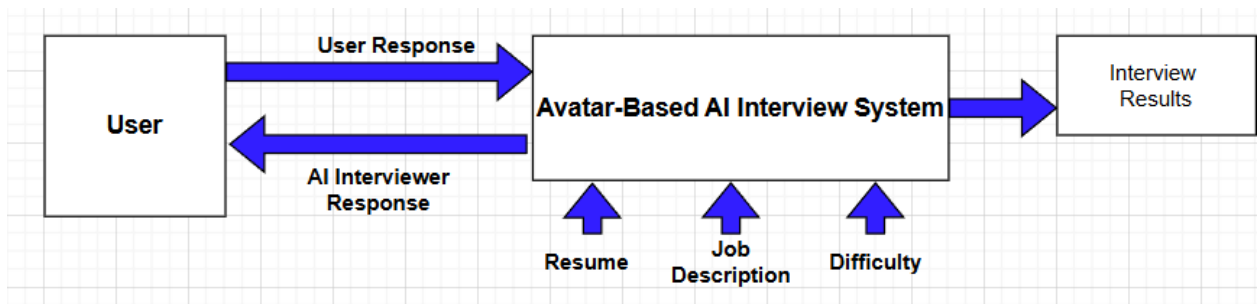


Figure 5: Data Flow Diagram Level-1

Components:

- **User:** The interviewee interacting with the system through voice.
- **Avatar-Based AI Interview System:** The core engine that processes user inputs and generates appropriate responses.
- **Inputs:**
 - **Resume (PDF):** Used to understand the candidate's background.
 - **Job Description:** Helps tailor the interview questions.
 - **Difficulty Level:** Adjusts question complexity.
- **Outputs:**
 - **Interview Results:** Final evaluation/report generated after the session.
 - **AI Interviewer Response:** Dynamic responses generated by the AI and sent back to the user.
 - **User Response:** Spoken inputs from the user sent to the system for processing.

Flow Summary:

1. The system receives three main inputs: resume, job description, and difficulty level.
2. Based on this, the avatar-based AI system initiates an interview session.
3. During the interaction, the user responds, and the system processes the input to give an AI-generated reply.
4. After the session, final **Interview Results** are produced and stored/displayed.

Data Flow Diagram – Level 2 (DFD Detailed)

This L2 diagram dives deeper into the **internal components and services** of the system, showing how user interaction is processed and rendered.

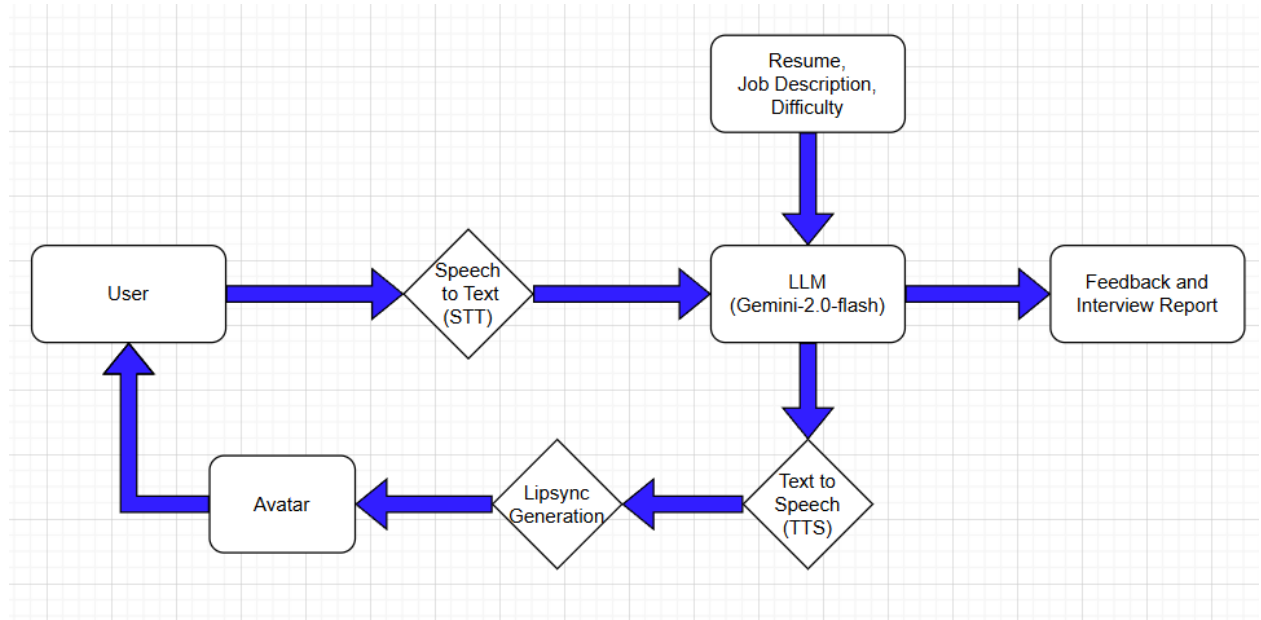


Figure 6: Data Flow Diagram level-2

Flow Explanation:

1. **User** speaks into the system.
2. The voice is processed by the **Speech-to-Text (STT)** module to convert audio input into text.
3. The **LLM (Gemini-2.0-flash)** receives the text response along with static inputs:
 - Resume
 - Job Description
 - Difficulty level
4. The LLM processes this data and:
 - Generates a **textual response** for the interviewer.
 - Stores feedback and generates the **Interview Report**.
5. The textual response is converted to speech by the **Text-to-Speech (TTS)** module.
6. The response is further passed through a **Lip-sync Generation** module to animate an **Avatar**, giving a realistic AI interviewer feel.
7. The Avatar's animated reply is sent back to the **User**.
8. The user continues the loop, completing the interaction cycle.

4.2.4 Sequence Diagram

The sequence diagram illustrates the interaction between components in a time-sequenced manner, showing how a user's input flows through the system to produce the final interview evaluation.

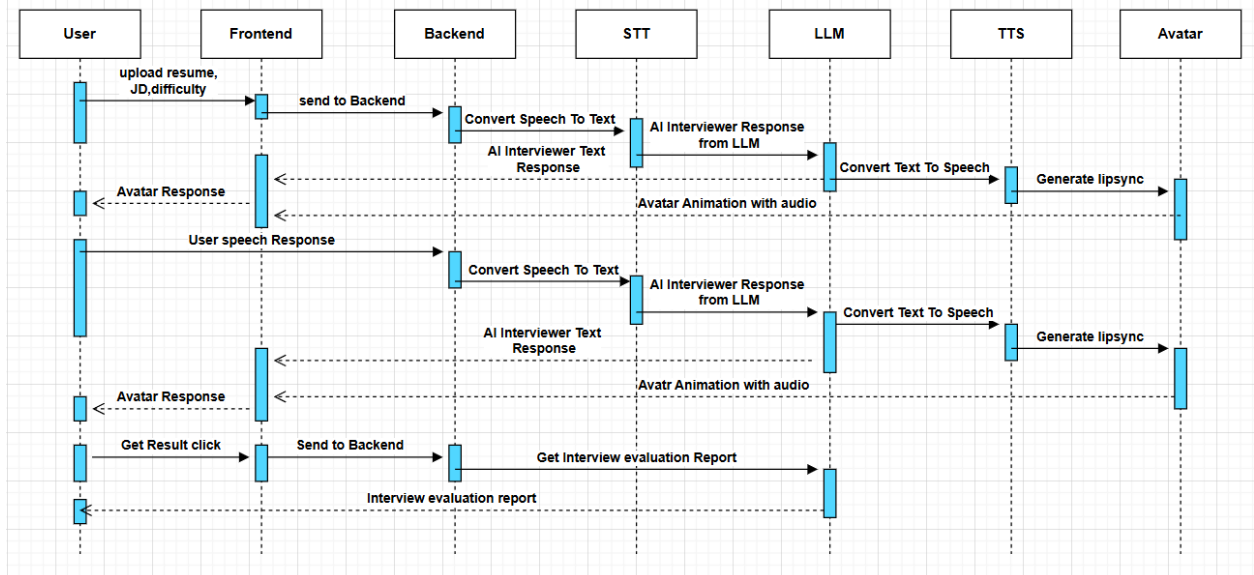


Figure 7: Sequence Diagram

Description of Workflow:

- The interaction begins when the user uploads their resume, job description, and selects the desired difficulty level on the frontend interface.
- This information is sent from the frontend to the backend for processing.
- The backend uses the speech-to-text module if needed and forwards the details to the LLM to generate appropriate interview questions.
- The generated interview question is then passed to the text-to-speech system which converts the text to audio format.
- The TTS output is used by the avatar module to generate lip-synced animations and simulate an interviewer asking the question.
- The avatar presents the question to the user through audio and animation.
- The user responds by speaking into the system.
- The user's speech is captured and passed from the frontend to the backend.
- The backend utilizes the speech-to-text system to transcribe the user's spoken response into text.
- The transcribed response is then evaluated by the LLM, which generates feedback and

possibly the next follow-up question.

- The feedback or next question is again converted to audio via the TTS engine and animated using the avatar system.
- This loop continues for a predefined number of questions or until the user stops.
- Once the interview is complete, the user clicks on the "Get Result" button.
- The frontend sends the result request to the backend.
- The backend collects the complete interview evaluation report, including scores and feedback from the LLM.
- The evaluation report is returned to the frontend and displayed to the user.

Key Modules Involved:

- **User:** Initiates the interaction by uploading inputs and responding via speech.
- **Frontend:** Acts as the interface for user interaction and coordinates communication with the backend.
- **Backend:** Handles business logic, calls external modules like LLM, STT, and TTS, and manages evaluation.
- **STT (Speech-to-Text):** Converts spoken audio input from the user into text format.
- **LLM (Large Language Model):** Generates questions, evaluates user responses, and provides scores and feedback.
- **TTS (Text-to-Speech):** Converts textual questions and responses into audio.
- **Avatar:** Animates and speaks the questions and responses using lipsync to simulate a realistic interview experience.

This sequence enables a seamless and interactive AI-driven mock interview that mimics real-world scenarios, helping users prepare effectively for job interviews.

4.2.5 Class Diagram

The class diagram outlines the object-oriented structure of the avatar-based AI interview system. It highlights the key classes, their attributes, and methods, and demonstrates how different components interact with each other to conduct a realistic AI-powered mock interview.

Description of Major Classes:**1. User**

This class represents the initial user who interacts with the system.

Attributes:

- name: Name of the user.

- resume: Uploaded resume document.
- job_description: The job description for the role the user is preparing for.
- difficulty_level: Selected difficulty level for the interview.

Methods:

- startInterview(): Begins the interview process.
- speakResponses(): Captures and processes the user's spoken answers.

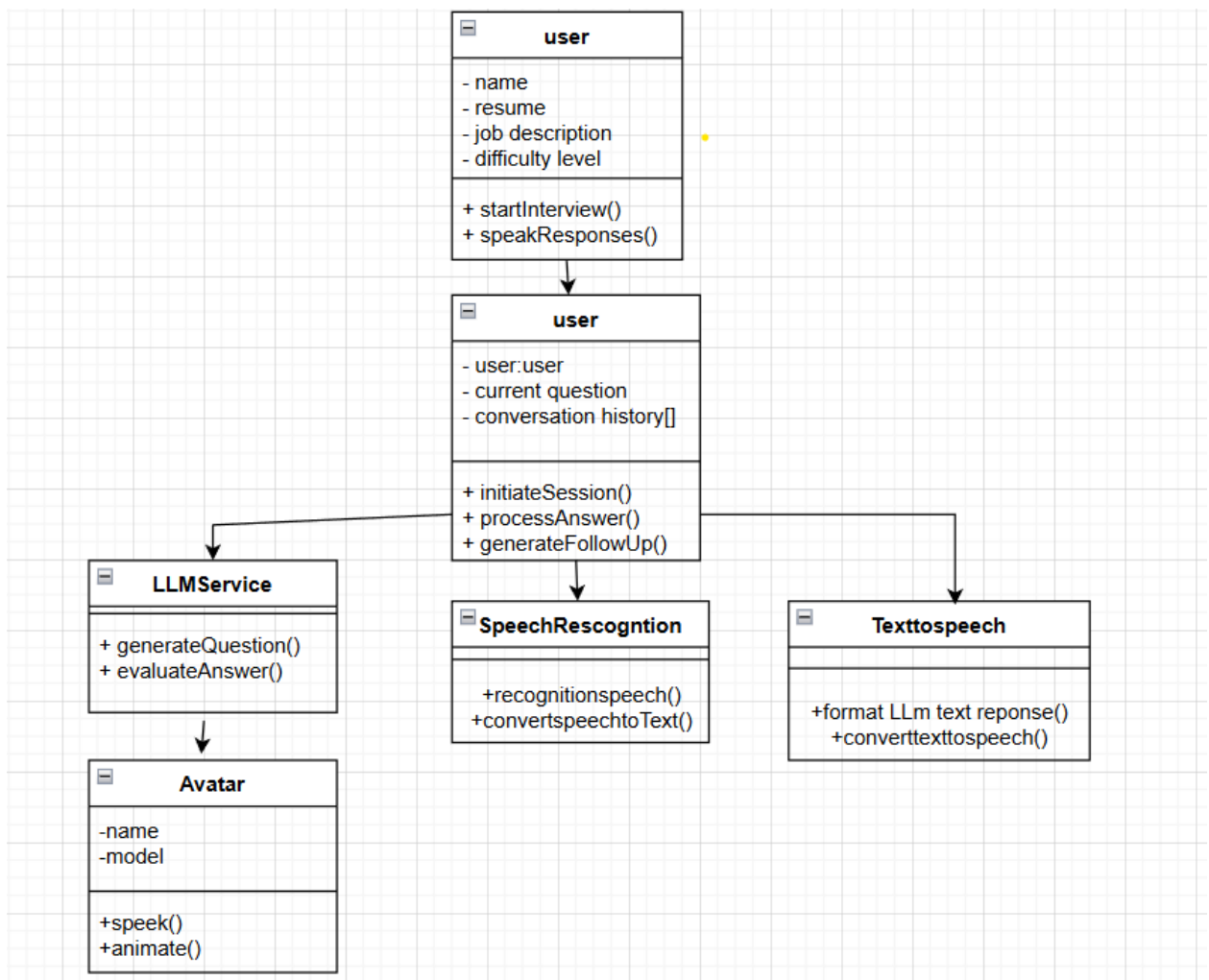


Figure 8: Class Diagram

2. InterviewSession

Handles the logic for an ongoing interview session.

Attributes:

- user: Reference to the User object.
- current_question: The interview question currently being asked.
- conversation_history[]: Array maintaining the conversation flow between user and system.

Methods:

- `initiateSession()`: Initializes a new interview session.
- `processAnswer()`: Handles the user's response and triggers evaluation.
- `generateFollowUp()`: Requests a follow-up question from the LLM based on the response.

3. LLMService

This service class connects to a large language model like GPT to generate and evaluate questions.

Methods:

- `generateQuestion()`: Creates a new question based on job description and difficulty.
- `evaluateAnswer()`: Evaluates the user's response and provides feedback or next steps

4. SpeechRecognition

Manages the transcription of spoken input.

Methods:

- `recognitionSpeech()`: Activates the microphone and records speech.
- `convertSpeechToText()`: Converts the captured speech into textual format.

5. TextToSpeech

Handles the conversion of text to audio output.

Methods:

- `formatLLMTextResponse()`: Prepares the LLM-generated text for vocal output.
- `convertTextToSpeech()`: Converts formatted text to audio.

6. Avatar

This class represents the animated interviewer component.

Attributes:

- `name`: Name or label for the avatar.
- `model`: Reference to the 3D or 2D avatar model used.

Methods:

- `speak()`: Outputs audio using avatar's voice model.
- `animate()`: Provides lip sync and facial animation for realism.

CHAPTER 5

IMPLEMENTATION

5.1 Modules

5.1.1 User Management Module

Purpose:

The **User Management Module** handles the collection, storage, and processing of user data, as well as the initialization of the mock interview session. This module ensures that the system personalizes the interview based on user inputs, like resume, job description, and difficulty level.

Detailed Explanation:

- **Data Collection:** The user enters their personal details, uploads their resume, and provides the job description for the role they're preparing for. The user also selects the difficulty level of the mock interview.
- **Session Management:** Once the data is collected, the module triggers the start of the interview session, passing the necessary data to other modules (e.g., LLM for question generation).
- **Personalization:** The data collected from the user is used to tailor the interview questions and evaluation criteria, ensuring that the mock interview aligns with the user's career goals.

5.1.2 Interview Session Module

Purpose:

The **Interview Session Module** is responsible for managing the flow of the mock interview, coordinating between different components like the user interface, backend, and AI systems.

Detailed Explanation:

- **Session Initialization:** The interview session starts with the **User Management Module** triggering the session. It provides the initial data (resume, job description, difficulty level) to the backend for processing.
- **Session Flow Control:** This module manages the entire flow of the interview, including controlling the questions and responses. It ensures that the interview process remains smooth and that the system adapts based on the user's answers.

-
- **Interaction with Other Modules:** The module interacts with the LLM to generate questions, handles the responses, and ensures that the interview continues logically from one question to the next, including follow-up questions.

5.1.3 LLM Service Module

Purpose:

The **LLM Service Module** uses a **large language model** (like **Gemini 2.5** or **GPT-3**) to generate interview questions, evaluate answers, and even provide follow-up questions. This module forms the backbone of the mock interview process.

Detailed Explanation:

- **Question Generation:** Based on the user's resume, job description, and the difficulty level selected, the LLM generates personalized, context-aware interview questions. These questions are designed to mimic real-world job interviews.
- **Response Evaluation:** After the user responds, the LLM evaluates the relevance and quality of the answer, providing feedback if necessary. The model can also assign a score based on answer quality, fluency, and context.
- **Follow-Up Questions:** The LLM analyzes the user's response and generates follow-up questions or feedback. This simulates a real interview scenario where interviewers ask for more details or clarification.

5.1.4 Speech Recognition (STT) Module

Purpose:

The **Speech Recognition Module** converts the user's spoken responses into text that can be processed by the system. This is an essential part of making the mock interview interactive and voice-based.

Detailed Explanation:

- **Speech-to-Text Conversion:** The module listens to the user's spoken response and converts it into text using a speech recognition engine like **Google Web Speech API** or **Whisper**.
- **Real-Time Interaction:** The user speaks their answer to an interview question, and the system processes it in real time. The converted text is then sent to the backend for further processing (i.e., evaluation by the LLM).
- **Accuracy and Adaptability:** The STT module must be capable of handling noisy environments, different accents, and various speech patterns. It should also provide high

accuracy for real-time transcription to ensure the system evaluates the correct response.

5.1.5 Text-to-Speech (TTS) Module

Purpose:

The **Text-to-Speech Module** converts textual information into audio, enabling the system to interact with the user through spoken language. This is important for creating a **realistic avatar interaction**.

Detailed Explanation:

- **Speech Generation:** Once the LLM generates a response or follow-up question, the TTS module converts the generated text into audio.
- **Voice Selection and Synthesis:** The system uses TTS engines (such as **Edge TTS** or **Google TTS**) to synthesize natural-sounding voice output. The quality of the TTS system directly impacts the user experience, ensuring the audio is clear and understandable.
- **Real-Time Audio Output:** The TTS system synchronizes with the avatar to ensure the speech is well-timed with lip-sync animations. It enables the avatar to speak in a human-like manner during the interview.

5.1.6 Avatar Interaction Module

Purpose:

The **Avatar Interaction Module** provides the user with a virtual interviewer, displaying the interview questions and responses in the form of a 3D avatar with animated facial expressions and lip sync.

Detailed Explanation:

- **Avatar Representation:** The module renders the avatar that interacts with the user. It can be a 3D or 2D model depending on your system's implementation. The avatar serves as the visual component that communicates with the user.
- **Lip Sync and Animation:** The avatar's lip sync is synchronized with the TTS-generated audio. The avatar also uses **Rhubarb Lipsync** or **Mixamo** animations to simulate realistic facial expressions and body language based on the interview context.
- **User Engagement:** The avatar enhances the experience, making the mock interview more immersive and engaging. It also plays a critical role in delivering questions and feedback to the user.

Language Model Processing & Intelligence

The core of the backend logic utilizes the Groq API with the llama-3.1-8b-instant model to handle intelligent requests.

Contextual Memory: We implement ConversationBufferMemory to maintain session-state and conversation history, ensuring that the AI has full context of the user's project journey.

Dynamic Prompt Engineering: The system uses a create_system_prompt method that applies difficulty modifiers (Easy, Medium, Hard) to tailor technical responses and problem-solving tasks.

```

class LanguageModelProcessor:
    def __init__(self):
        groq_api_key = os.getenv("GROQ_API_KEY")
        if not groq_api_key:
            raise RuntimeError(
                "Missing GROQ_API_KEY. Set it in your environment or in back-end/.env"
            )
        self.llm = ChatGroq(
            model="llama-3.1-8b-instant",
            api_key=groq_api_key,
        )

        # Memory to hold conversation history
        self.memory = ConversationBufferMemory(memory_key="chat_history", return_messages=True)
        self.conversation = None # Placeholder for the conversation chain
        self.resume=None
        self.job_description=None
        self.difficulty=None

    @staticmethod
    def extract_text_from_pdf(pdf_file_obj):
        try:
            reader = PyPDF2.PdfReader(pdf_file_obj)
            text = "".join([page.extract_text() for page in reader.pages])
            return text.strip()
        except Exception as e:
            import traceback
            traceback.print_exc()
            return f"Error reading PDF: {e}"

    def create_system_prompt(self, resume, job_description, difficulty_level):
        # Adjust the system prompt based on difficulty level
        difficulty_modifiers = {
            "Easy": "Focus on basic and general questions.",
            "Medium": "Include moderately technical and problem-solving questions.",
            "Hard": "Ask deep, technical, and thought-provoking questions requiring detailed responses."
        }

```

5.1.7 Follow-Up Generation Module

Purpose:

The **Follow-Up Generation Module** dynamically adjusts the flow of the interview based on user responses, generating appropriate follow-up questions.

Detailed Explanation:

- **Context Awareness:** This module ensures that the system remains contextually aware of previous answers. For example, if a user mentions a specific skill or project, the follow-up question may inquire deeper into that subject.
- **Adaptive Interviewing:** It uses LLM (e.g., Gemini 2.5) to generate follow-up questions based on the user's response. This simulates a natural interview conversation, where the interviewer may ask for further details or clarification.
- **Realism:** The goal is to mimic the natural progression of a real interview, ensuring the system doesn't follow a rigid script but rather adapts to the flow of conversation.

5.1.8 Evaluation & Feedback Module

Purpose:

The **Evaluation & Feedback Module** analyses the user's responses, assigns scores, and provides constructive feedback to help the user improve.

Detailed Explanation:

- **Answer Analysis:** The module assesses the quality of the user's answers based on factors such as relevance, clarity, confidence, and completeness. This can be achieved using AI models to analyze text and tone.
- **Scoring:** The system assigns a score to the user's performance, simulating real-world evaluation criteria. This could include scores for different categories like communication skills, problem-solving ability, and technical knowledge.
- **Constructive Feedback:** The system provides feedback based on the analysis, offering suggestions for improvement, such as tips on how to improve responses, handle difficult questions, or refine body language.

5.2 Technologies used

5.2.1. Large Language Model (LLM) – Gemini 2.5 Flash

- The system uses Gemini 2.5 Flash, a powerful large language model, for generating interview questions, follow-up prompts, and evaluating user responses.
- This model was chosen for its fast response time, contextual awareness, and natural language generation capabilities.
- It helped simulate realistic interviewer behavior by adapting to the user’s resume, job description, and selected difficulty level.

5.2.2. Speech-to-Text (STT) – Google Web Speech API

- Google Web Speech API was used to convert the user’s spoken answers into text in real time.
- This browser-supported API was selected for its ease of integration and accurate transcription capabilities.
- It enabled hands-free interaction, mimicking how users respond in real-world remote or AI-driven interviews.

5.2.3. Text-to-Speech (TTS) – Edge TTS

- Edge TTS was implemented to convert AI-generated text responses into natural-sounding audio.
- This allowed the avatar to “speak” with fluidity and expression, enhancing the realism of the system.
- Edge TTS was chosen for its high audio quality, minimal latency, and compatibility with lip-sync mechanisms.

5.2.4. Avatar System – Ready Player Me (RPM)

- Ready Player Me was used to integrate a 3D avatar that acts as the virtual interviewer.
- The avatar delivered AI-generated questions and feedback using synced facial animations and lip movement.
- This provided users with a more engaging and immersive experience, closely resembling face-to-face interviews.

5.2.5. Frontend Technologies – React.js and Three.js

- The frontend was developed using **React.js**, a modern JavaScript framework that enabled fast, component-based UI rendering.
- React provided a dynamic structure for handling user interactions, form validations, and avatar integration.

- **Three.js** was used alongside React to render and animate the 3D avatar within the browser environment, ensuring an immersive and responsive user experience.

5.2.6. Backend Technologies – Python and FastAPI

- The backend was developed using **Python**, with **FastAPI** as the web framework to build asynchronous APIs.
- FastAPI was selected for its speed, simplicity, and modern design, making it ideal for handling real-time data processing between frontend, LLM, STT, and TTS services.
- The backend managed session flow, routed data between modules, and invoked external APIs for AI services and speech processing.

CHAPTER 6

TESTING

Each major module of the Avatar-Based AI Mock Interview System was tested independently to ensure proper functionality and seamless integration. The following are the results and observations from testing each section:

6.1 TYPES OF TESTING

6.1.1 Resume, Job Description, and Difficulty Input

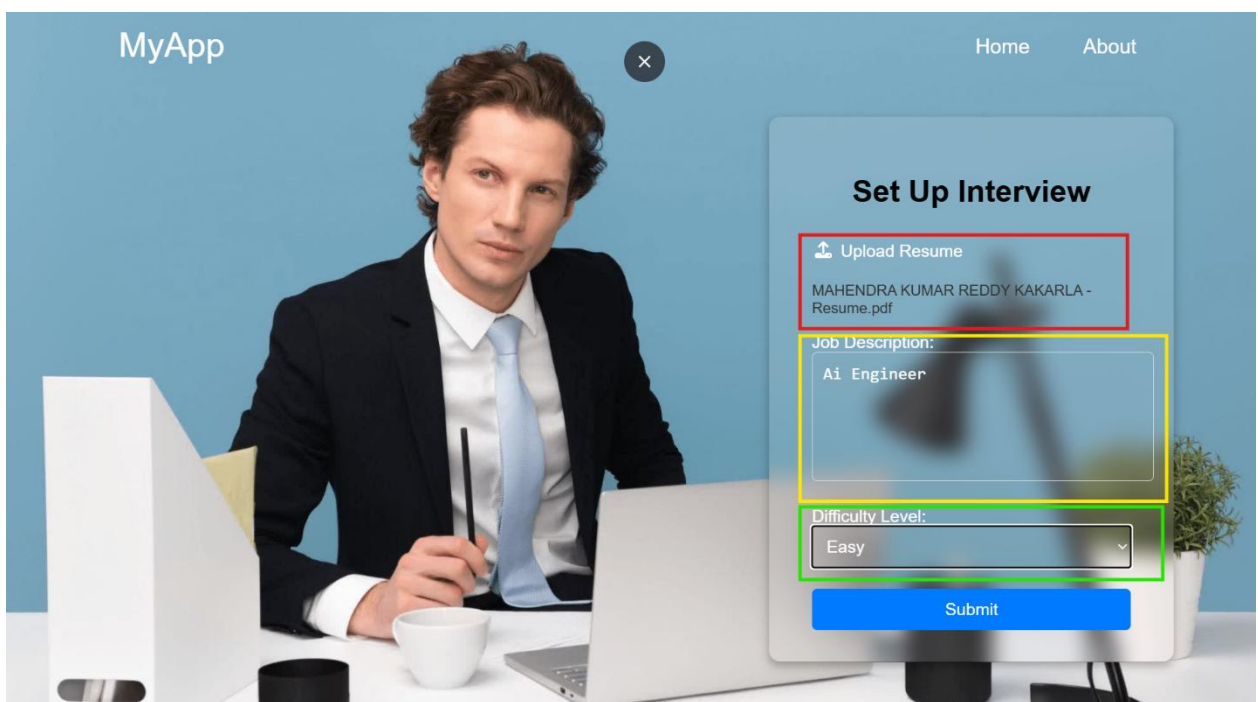


Figure 9: Input section Test results.

- **Module:** User Input Section
- **Objective:** To ensure that the system correctly receives and validates all required user inputs before proceeding to the interview session.
- **Test Steps:**
 1. Open the system's homepage/input interface.
 2. Upload a valid resume file (e.g., .pdf format).
 3. Enter a job description text.
 4. Select a difficulty level from the dropdown.
 5. Submit the inputs.

- **Expected Output:**

The system should validate all fields, accept supported file types, and navigate to the interview panel if all inputs are correct.

- **Actual Output:**

The system accepted .pdf, .docx, and .txt formats. Any missing or invalid fields prompted validation messages. The interface navigated smoothly to the interview panel.

- **Explanation:**

This test confirms that the input handling logic works as intended and that incorrect inputs are appropriately blocked before reaching the backend. It also confirms that the data is packaged and passed forward to trigger the AI question generation.

6.1.2 Interview Panel Functionality and Question Delivery

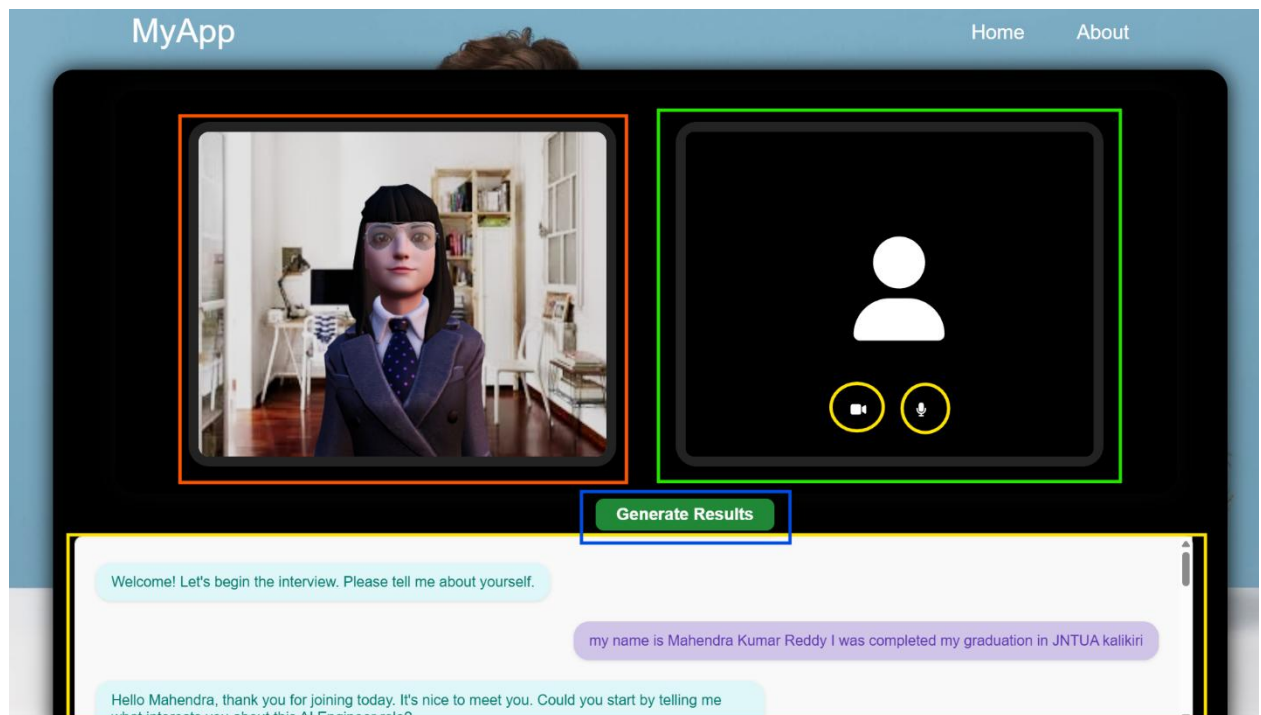


Figure 10: Interview panel test results.

- **Module:** Interview Panel
- **Objective:** To test whether the system can correctly display and speak questions using the avatar after receiving inputs.
- **Test Steps:**
 1. Complete the input step and proceed to the interview panel.
 2. Observe how the first question is presented.
 3. Monitor the avatar's lip movement, voice output, and real-time syncing.

- **Expected Output:**

The system should fetch a question from the LLM, convert it to audio using TTS, and present it through the avatar with synchronized animation.

- **Actual Output:**

The question appeared on screen and was clearly spoken by the avatar with smooth animation. Delays were minimal.

- **Explanation:**

This test ensures the correct integration between the LLM module, TTS engine, and avatar component. It validates whether a realistic interview-like experience is delivered and maintained throughout.

6.1.3 Voice Response and STT Accuracy

- **Module:** Speech-to-Text (Google Web Speech API)

- **Objective:** To verify that the system captures the user's spoken input, converts it accurately to text, and sends it for AI processing.

- **Test Steps:**

1. When prompted by the avatar, speak a response into the microphone.
2. Ensure normal tone and language (English).
3. Wait for the system to transcribe and process the answer.

- **Expected Output:**

The voice input should be correctly captured and transcribed, with high accuracy, even in typical room conditions.

- **Actual Output:**

The transcription was mostly accurate. Minor misinterpretations occurred in fast or unclear speech but did not break the flow.

- **Explanation:**

This test evaluates the effectiveness of the STT module. Accurate transcription is critical since the response is evaluated by the LLM. The test also observes system behavior under slightly noisy conditions.

6.1.4 Result Evaluation and Score Display

- **Module:** Evaluation and Result Palette

- **Objective:** To confirm that the system evaluates responses and provides complete and clear feedback to the user after the interview ends.

- **Test Steps:**

1. Complete a full mock interview session (3–5 questions).
2. Click on the “Get Result” or “Finish Interview” button.
3. Observe the result screen.

- **Expected Output:**

The system should display scores, feedback (strengths and weaknesses), and conversation history clearly.

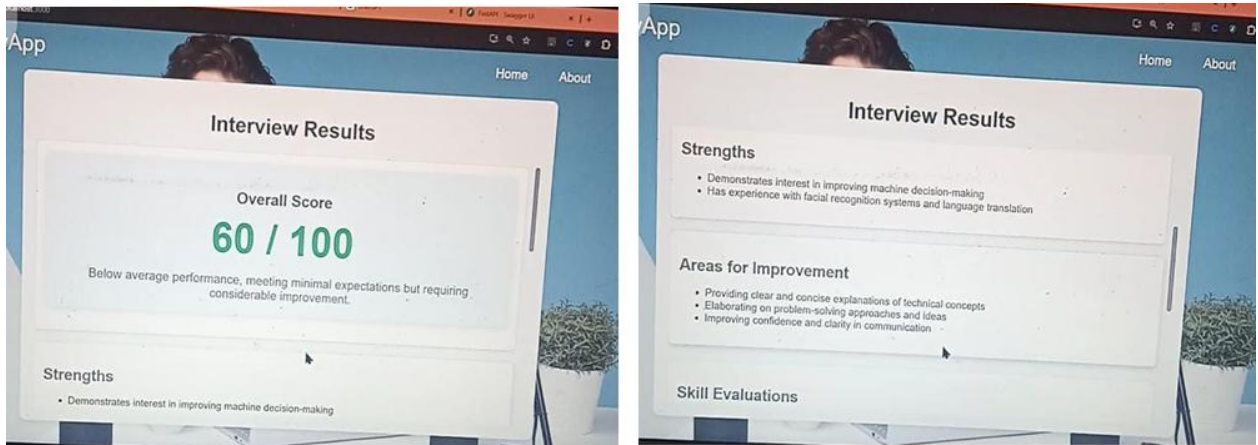


Figure 11: Interview Results report test results.

- **Actual Output:**

The result screen showed percentage-based scores, qualitative feedback, and a scrollable chat history. No errors were found in transitions or data rendering.

- **Explanation:**

This test ensures that the evaluation engine processes the user's entire interaction and generates a meaningful, accurate result summary. It validates backend processing as well as frontend rendering and user experience.

CHAPTER 7

RESULTS

The system was tested thoroughly across all major functional modules to validate its reliability, responsiveness, and accuracy. Below are the consolidated results based on the testing activities performed on each core module:

Input Section

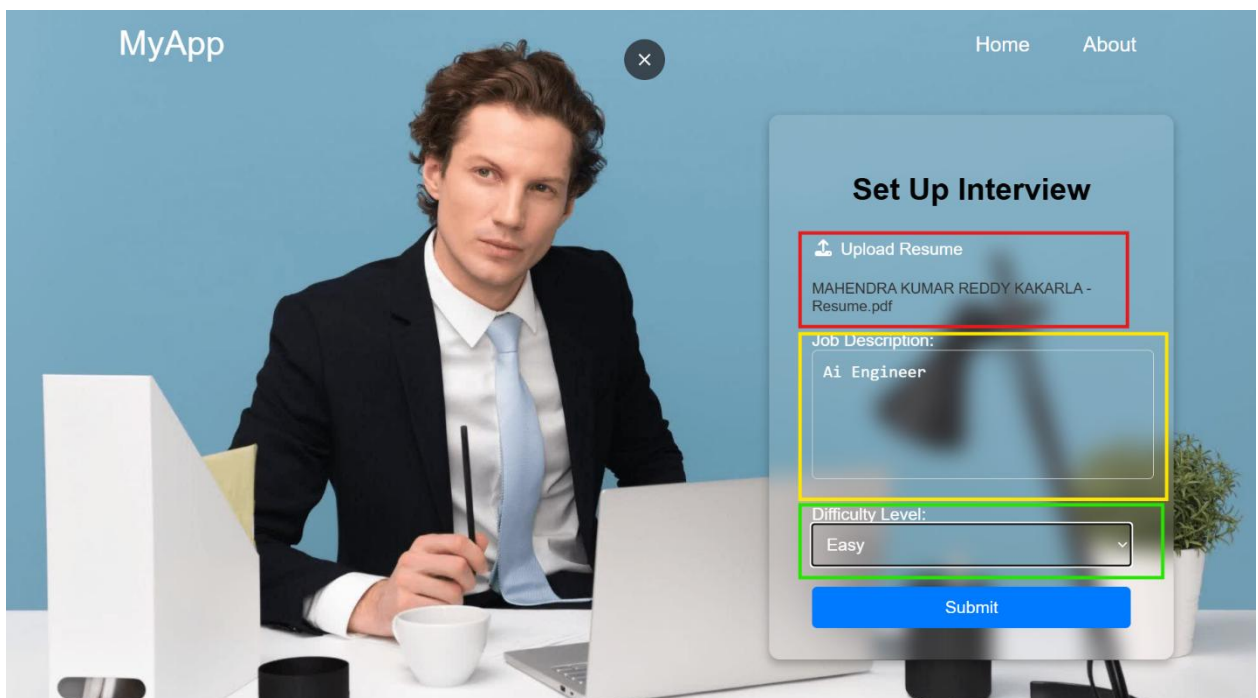


Figure 12: Input section results.

The first interface screenshot displays the input section of the system, where the user is prompted to upload their resume, provide a job description, and select the desired difficulty level. During testing, this section was found to be intuitive and responsive. The file upload feature functioned reliably across multiple formats, and real-time validation messages ensured that no field was left incomplete. The dropdown for difficulty selection updated and passed values correctly. Once all fields were populated, the “Start Interview” button transitioned the user smoothly into the next stage. No functional or UI-level errors were encountered during multiple test runs, confirming the reliability of the input module.

Interview Panel

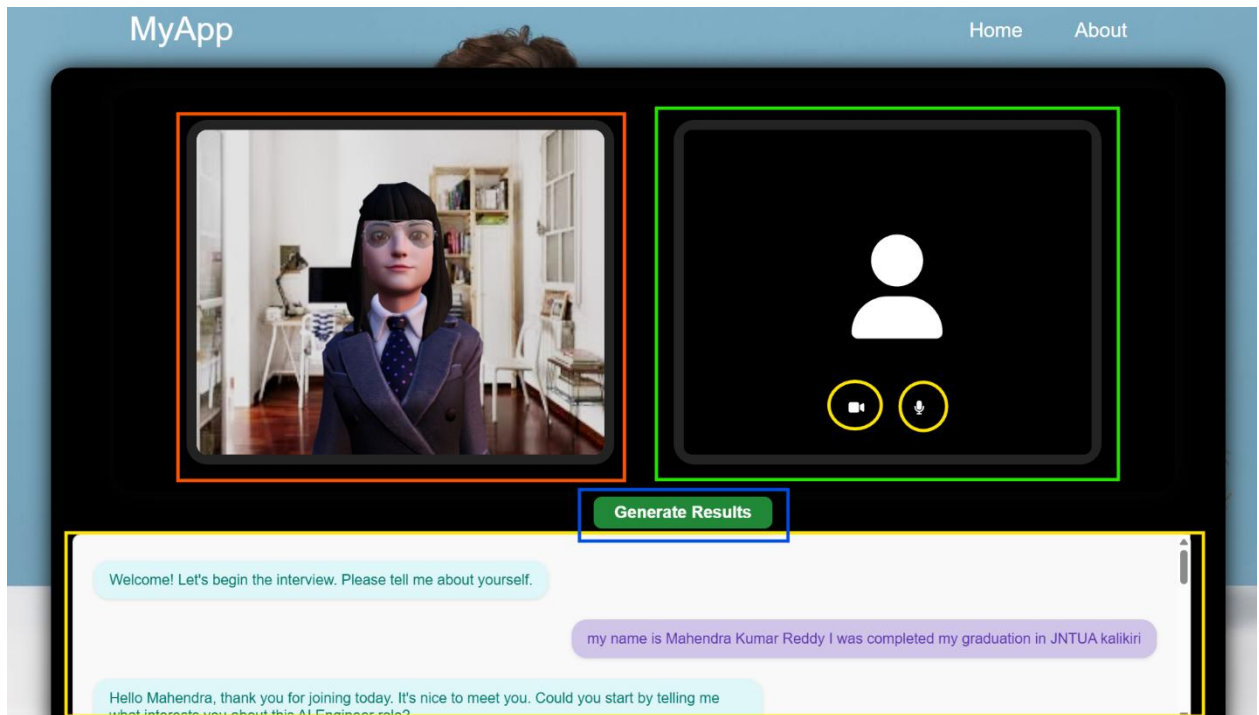


Figure 13: Interview panel results.

The second image represents the main interview panel, where AI-generated questions are presented and the avatar begins interacting with the user. Testing confirmed that the transition from the input screen to this panel was seamless. Questions generated by the LLM were clearly displayed on the screen, while the avatar simultaneously spoke them out loud using the integrated TTS module. The avatar's lip movement and facial expressions were synchronized with the audio, maintaining a natural flow throughout the session. The interface allowed users to pause, continue, or skip to the next question without delay. Overall, this stage provided a smooth and immersive experience, functioning reliably during all test sessions.

Speech Response Interaction

The third image captures the user's response phase during the interview session, where the system listens to and transcribes the spoken answer using the integrated Speech-to-Text (STT) engine. During testing, this module responded well to real-time input. As the user began speaking, the microphone activation was smooth, and the transcription appeared almost instantly on screen. The STT system handled various accents with acceptable accuracy and functioned well even in moderately noisy environments. The transcribed text was cleanly passed to the backend for evaluation without interrupting the flow of the interview. The live feedback and

voice handling were stable across repeated tests.

Result Palette

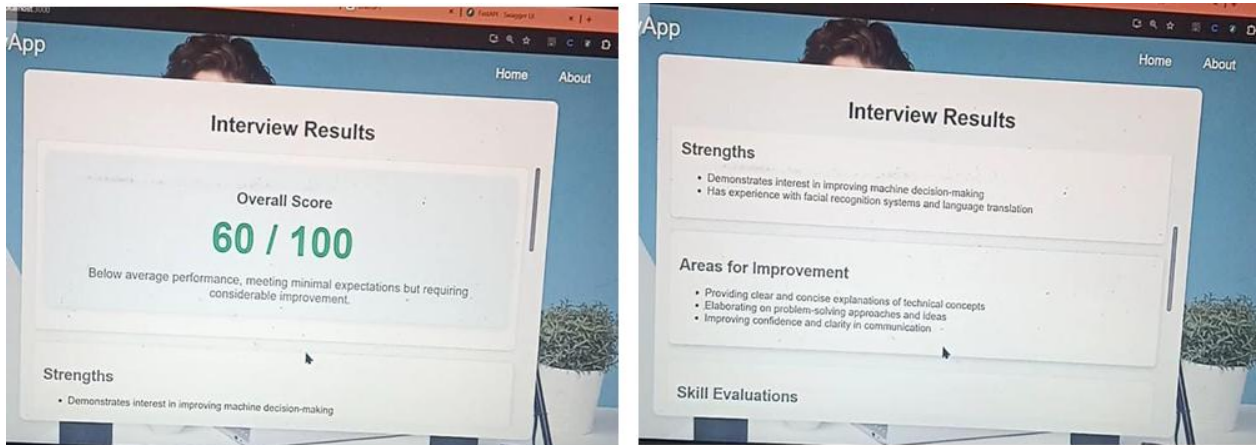


Figure 14: Results pallet result.

The final screenshot represents the result section, where the user receives a detailed evaluation after completing the mock interview. This module displayed the final performance score along with categorized feedback highlighting strengths and improvement areas. The feedback aligned well with the quality of the user's spoken answers. The result interface also included a chat-style review of the full interview session, allowing users to revisit their questions and answers. During testing, this section consistently loaded without errors, presented accurate results, and maintained a clean layout. It successfully provided the user with useful insight into their interview performance, completing the overall mock interview flow effectively.

CHAPTER 8

CONCLUSION

In conclusion, the **Avatar-Based AI Mock Interview System** successfully integrates multiple advanced technologies, including large language models (LLMs), speech recognition, text-to-speech synthesis, and 3D avatars, to create a realistic and interactive mock interview experience. The system allows users to practice job interviews in a controlled environment, helping them improve their communication skills and gain confidence before facing actual interviews. The seamless integration of these components ensures a smooth and engaging user experience, from input submission to feedback presentation.

The system has been thoroughly tested across all its components, with each module functioning as expected under various conditions. Whether handling user input, processing speech, generating AI-driven questions, or providing feedback through an avatar, the system performed reliably in all stages of the mock interview. The successful integration of speech-to-text and text-to-speech technologies allowed for natural, real-time interaction between the user and the AI avatar, while the final feedback mechanism provided valuable insights for improvement.

While the project has met its objectives of creating a functional and immersive AI mock interview system, future improvements can be made to further enhance the user experience. This could include expanding language support, refining AI evaluation algorithms, or introducing more sophisticated emotional analysis during the interview. Overall, the project serves as a powerful tool for job seekers, providing them with a personalized and dynamic interview preparation experience.

CHAPTER 9

FUTURE SCOPE

The **Avatar-Based AI Mock Interview System** serves as a highly functional tool for job interview preparation, but there are several areas for enhancement and future development that could further extend its utility and capabilities. Below are some potential directions for future work on this project:

1. **Multi-Language Support:**

Expanding the system's capabilities to support multiple languages would make it accessible to a wider audience. By incorporating additional language models and TTS engines, users could practice interviews in various languages, tailoring the experience to their linguistic preferences and needs.

2. **Emotional and Sentiment Analysis:**

Integrating sentiment analysis into the system would allow it to assess not just the content of the user's responses but also the tone and emotional cues in their speech. This would help provide more accurate feedback on a candidate's confidence, enthusiasm, and communication style, which are often important factors in real interviews.

3. **Real-Time Interviewer Behaviour Adaptation:**

Currently, the system generates predefined questions. However, the integration of adaptive AI that can change its behaviour based on user responses would make the system more realistic. For example, if a user gives a particularly strong or weak answer, the system could adjust the difficulty or nature of follow-up questions, mimicking a real interviewer's responses.

4. **Advanced User Analytics and Insights:**

The system can be enhanced with more sophisticated analytics that tracks user performance across multiple sessions. It could then provide personalized recommendations based on past performance, such as suggesting areas for improvement or identifying commonly made mistakes during interviews.

5. **Virtual Interviewer with AI Personality:**

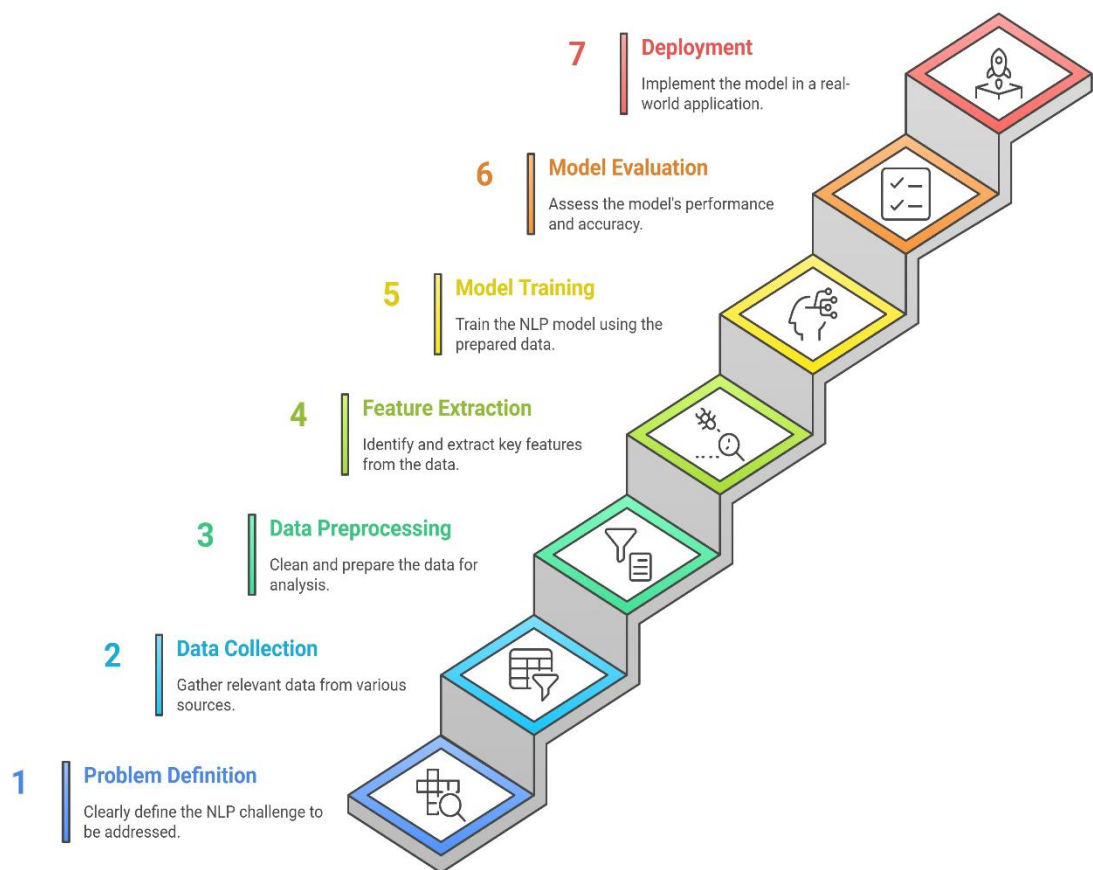
Another exciting development would be to create a virtual interviewer with a unique personality, offering users a chance to practice with different styles of interviewers. This could include varying degrees of formality, friendliness, or even stress-inducing

behaviour to help users better prepare for diverse real-world interview situations.

6. Integration with Real Job Platforms and HR Systems:

In the future, integrating the mock interview system with real job platforms or HR management systems could allow users to receive feedback from real interviewers in a simulated environment. This could involve using AI-generated avatars to simulate interviews for specific companies or industries, offering job-specific feedback and training.

Steps to Solve NLP Problem



CHAPTER 10

REFERENCES

- [1] M. J. Carey, D. Nguyen, and A. Gupta, “Conversational Agents in AI-Powered Training Simulations: A Review of Intelligent Interview Systems,” *IEEE Access*, vol. 9, pp. 145321–145334, 2021, doi: 10.1109/ACCESS.2021.3129990.
- [2] S. Zhang, X. Liu, and Y. Wang, “Development of Emotion-Aware Virtual Avatars for Realistic Human-Computer Interaction,” *IEEE Transactions on Affective Computing*, vol. 13, no. 2, pp. 1021–1034, 2022, doi: 10.1109/TAFFC.2021.3056489.
- [3] J. Lee, H. Kim, and K. Yoon, “Real-Time Speech-to-Text Conversion for Human-Computer Interaction Using Google Web Speech API,” *IEEE Internet of Things Journal*, vol. 8, no. 7, pp. 5692–5703, Apr. 2021, doi: 10.1109/JIOT.2021.3057610.
- [4] Ghosh and P. Arora, “AI-Based Job Interview Evaluation System Using NLP and Machine Learning,” *IEEE Intelligent Systems*, vol. 36, no. 5, pp. 36–45, Sep.–Oct. 2021, doi: 10.1109/MIS.2021.3081402.
- [5] L. Chen, M. Qiu, and S. Zhang, “Web-Based Virtual Agents for Interactive Learning and Assessment,” *IEEE Transactions on Learning Technologies*, vol. 14, no. 3, pp. 253–262, Jul.–Sep. 2021, doi: 10.1109/TLT.2020.2998410.
- [6] K. B. Malik, R. Singh, and A. Sharma, “Enhancing User Engagement with AI Avatars in EdTech Platforms,” *IEEE Transactions on Human-Machine Systems*, vol. 52, no. 1, pp. 98–108, Feb. 2022, doi: 10.1109/THMS.2021.3103895.
- [7] T. Brown et al., “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.